

StudyBuddy

kushal reddy& Abhinav Kumar

IE:5374

Contents

Introduction.....	3
System Architecture.....	3
Components and contracts.....	5
Model, Route & Retrieval	6
Data	6
Adaptivity	6
Demo (Results)	8
Limitations	9

Introduction

Studying is generally a passive and nondescriptive activity. Standard question banks ask the same questions regardless of learner's abilities. Most tools found online forget about a student's weak points across different sessions/attempts so time is spent re-visiting already learnt material. And reading again and underlining may feel productive but hardly reveals what a learner does not yet know. They can't set an exam on demand and provide answers from the public internet sources instead of the student's own lecture notes.

Instantly "StudyBuddy" fills these gaps. The student uploads study material (a pdf or pasted notes) and StudyBuddy extracts main ideas, generates quizzes of different levels of difficulty, grades responses with respect to the original content, keeps track of the weak points of each concept throughout the sessions and revisits the exact concepts the student failed at increasingly difficult levels as progression increases. In addition to answering open-ended questions on the topic, a conversational tutor agent can use tools to control the application (start a test, find areas of weakness, or clarify an idea). The more you use the technique the more useful it is. Each wrong answer influences the next question.

System Architecture

The core idea of StudyBuddy is that there are specialized agents: extraction, generation, grading, adaptation, and tutoring are different tasks that benefit from different prompts and, if applicable, different models. The tutor is an agentic LangChain layer on top, that allows the model to choose which tools to use, while the main learning loop is a deterministic LangGraph state machine. The project is built leveraging the LangChain ecosystem in its entirety, including: ChatOpenAI models with environment configuration, ChatPromptTemplate prompts, LCEL chains, `with_structured_output()` for typed results, Chroma retrieval, a `create_tool_calling_agent + AgentExecutor` tutor with conversation memory, and LangGraph for orchestration.

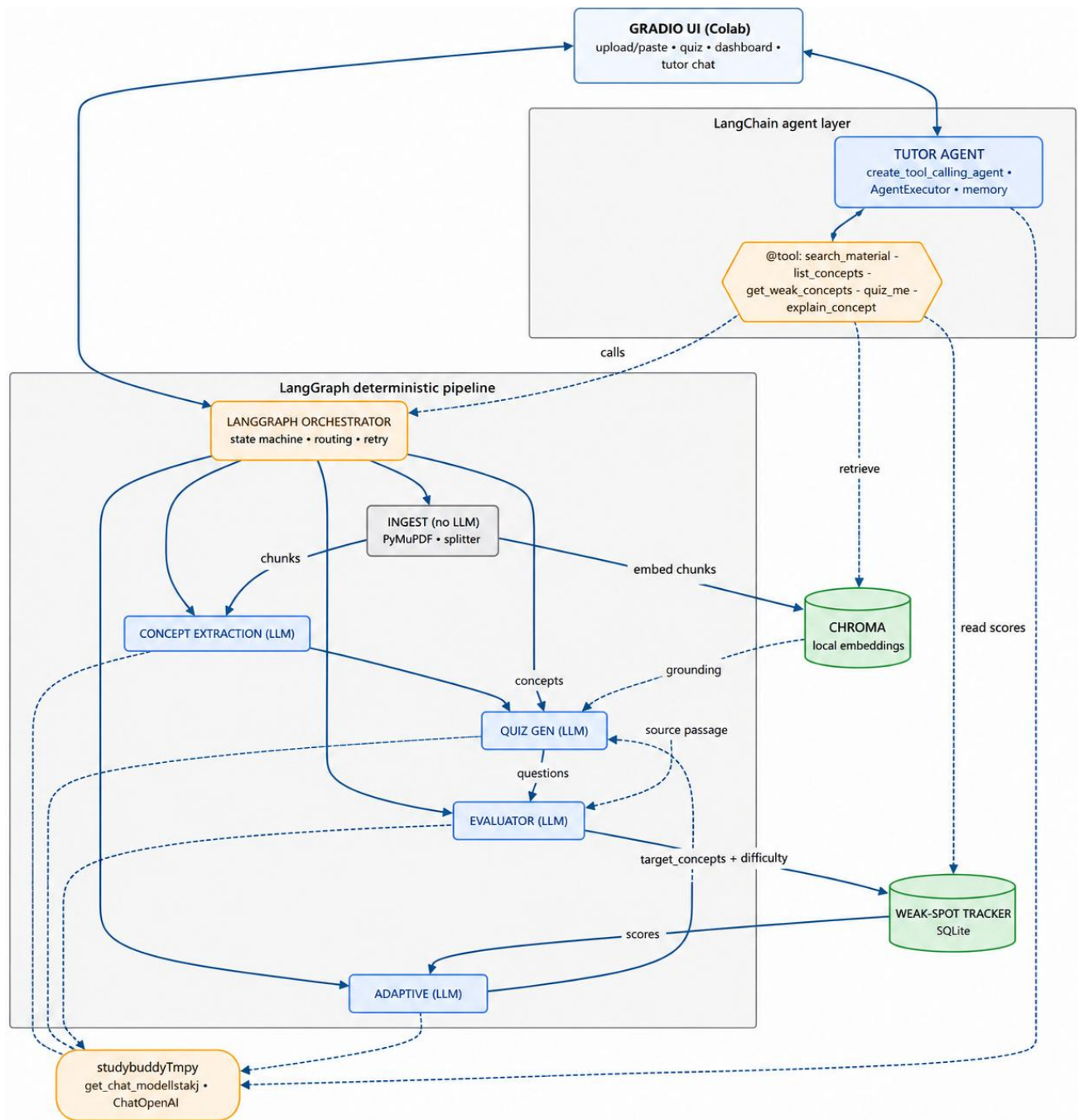


Figure 1: System architecture: one Chroma vector store, one SQLite store, and one env-routed model factory are shared by a fixed LangGraph pipeline (right) and an agent tutor layer (left).

Components and contracts

Component	Model	Function
Ingest	no LLM	Takes in PDFs or text files, splits them into topic-based sections using PyMuPDF and RecursiveCharacterTextSplitter, and stores them as embeddings in Chroma.
Concept Extraction	EXTRACTION_MODEL	Provides organized outputs by analyzing each chunk and extracting important phrases, definitions, and relationships using a specialized extraction model.
Quiz Generation	QUIZ_MODEL	Creates multiple-choice, true/false, or short-answer questions at three distinct difficulty levels by feeding the extracted concepts and background data into a quiz model.
Evaluator	EVAL_MODEL	Provides a conclusion and an explanation after comparing user responses to the original text using an evaluation model. This is done in code for multiple-choice questions and true/false; for short answers, it compares the answer to the source directly.
Weak Spot Tracker	no LLM	Uses SQLite to track user performance by concept, keeping an eye on tries, right answers, and streaks. There is no use of a linguistic model.
Adaptive	ADAPTIVE_MODEL	Uses the user's score table to generate a customized review plan. If the score is less than 60%, it suggests revisiting; if it is more than 80%, it raises the difficulty; if not, it maintains the same level of difficulty.
Tutor (tool calling)	TUTOR_MODEL	Manages chat messages and has the ability to summon up to 10 different tools in order to offer grounded responses or trigger app operations when necessary.
Orchestrator	router	Uses LangGraph's StateGraph to route jobs through the pipeline; if any model produces output that is not correct, it automatically retries.

Model, Route & Retrieval

The core idea of StudyBuddy is that there are specialized agents: extraction, generation, grading, adaptation, and tutoring are different tasks that benefit from different prompts and, if applicable, different models. The tutor is an agentic LangChain layer on top, that allows the model to choose which tools to use, while the main learning loop is a deterministic LangGraph state machine. The project is built leveraging the LangChain ecosystem in its entirety, including: ChatOpenAI models with environment configuration, ChatPromptTemplate prompts, LCEL chains, with_structured_output() for typed results, Chroma retrieval, a create_tool_calling_agent + AgentExecutor tutor with conversation memory, and LangGraph for orchestration.

Retrieval is local and grounded. Document chunks are stored in a persistent Chroma vector store, indexed by session and document information, and embedded using a local sentence-transformers model (all-MiniLM-L6-v2). This means that RAG keeps the demo CPU friendly, even if the chat gateway does not offer an embeddings endpoint. Both the quiz generator and the evaluator retrieve grounding texts in order to link questions and grades to the actual submitted content rather than to model priors.

Data

The pieces are embedded and stored in Chroma for later retrieval. SQLite stores structured progress and content. Durable documents, concepts, quizzes, and questions tables allow sessions to persist across restarts and allow study across multiple documents. A concept_scores table records the number of attempts, number correct, streaks, and current difficulty for each concept. A quiz_history table logs each question answered and a timestamp. From the dashboard, we score expertise from concept_scores, and bucket it into 3 bands: below 0.5 (red), below 0.8 (yellow), and 0.8 and above (green). This provides the learner a quick overview of where to focus and learn.

Adaptivity

The adaptive loop is a LangGraph StateGraph over a shared StudyState: ingest -> embed -> extract ideas -> generate quiz -> (await user) -> evaluate -> update tracker -> adjust -> back to quiz generation. To make the pipeline resilient to sporadic erroneous model outputs, each LLM node that outputs invalid structured output sets an error flag, retries a limited number of times before gracefully failing.

The system is named the “adaptable” system. After grading, the Adaptive agent checks the table of per concept score and produces a revision plan. It revisits the ideas with less than 60% accuracy, escalates ideas with more than 80% accuracy to a harder level and keeps other concepts in the same level.

The next round focuses on learner weaknesses because the target ideas and difficulty of the plan feed directly into quiz generation. Concepts that need to be reviewed are also brought to the forefront by the spaced-repetition scheduler (SM-2). Thus, the “next round” will focus on recent

errors as well as concepts that are at risk of being forgotten.

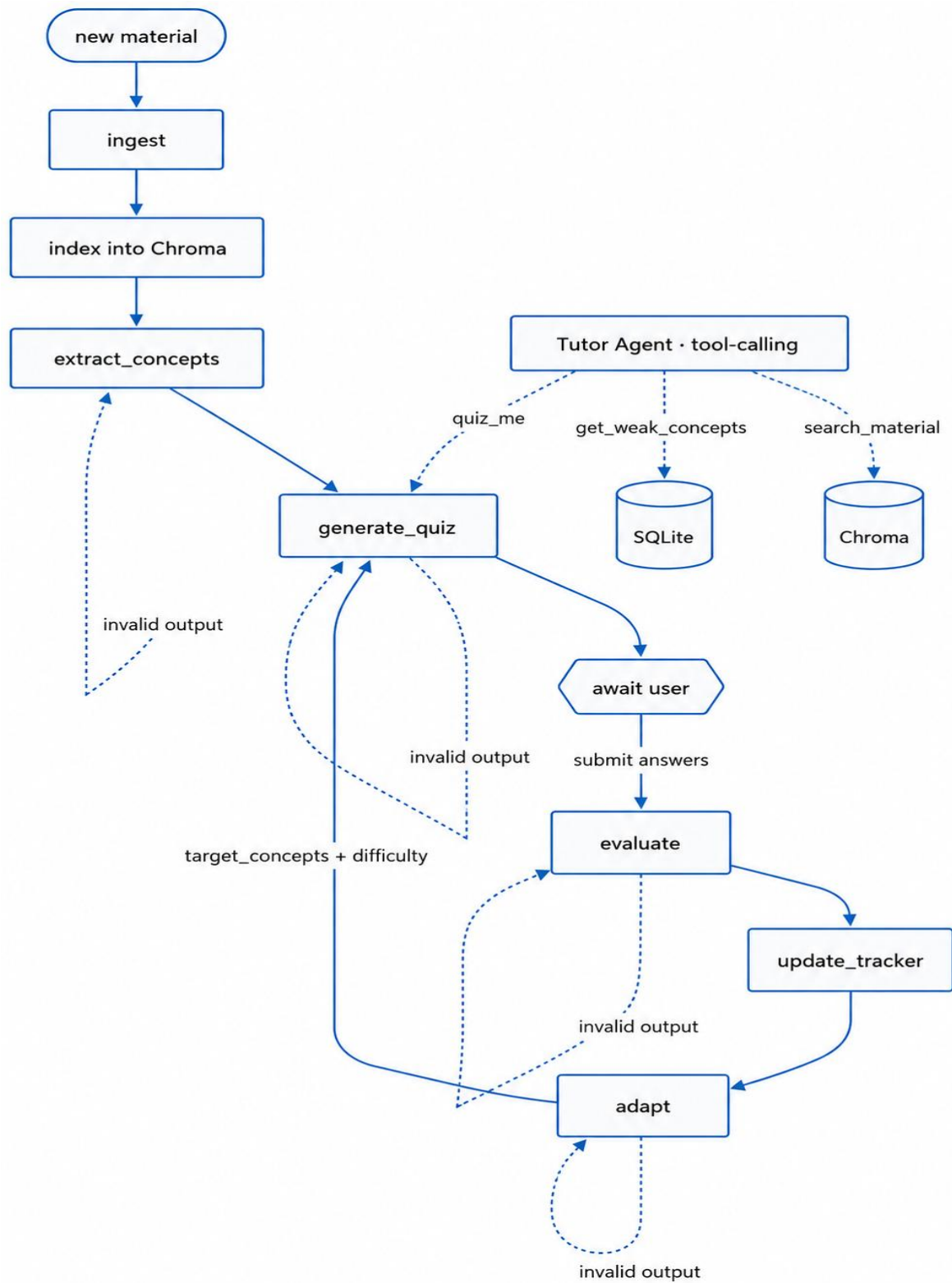


Figure 2: System architecture: an agent tutor layer (left) and a fixed LangGraph pipeline (right) share a Chroma vector store, a SQLite database, and an env-routed model factory.

Demo (Results)



Adaptive, agentic study assistant.

StudyBuddy is an educational aid. Generated concepts, questions, grades, and tutor answers can contain errors — verify against your source material.

Upload / Paste Quiz Flashcards Dashboard Tutor

Upload a PDF (with a text layer)

AttentionIsAllYouNeed.pdf 2.1 MB

...or paste study material

Ingest material

Each ingest **adds** a document — your session can hold several.

✓ Ingested **uploaded.pdf** — 116 concepts, 6 questions in round 1. Go to the **Quiz** tab.

Documents in this session

◦ **uploaded.pdf** (doc1)

Extracted concepts

- **English Constituency Parsing** — A specific task in natural language processing that involves analyzing the grammatical structure of sentences, presenting unique challenges for models.
- **Mixture-of-Experts Layer** — The sparsely-gated mixture-of-experts layer is a neural network architecture that activates only a subset of its components (experts) for each input, allowing for more efficient processing and scalability.
- **Attention Heads** — Attention heads are individual components of the attention mechanism that focus on different parts of the input, allowing the model to capture various aspects of the data simultaneously.
- **Imperfect Law** — The law is inherently flawed and cannot achieve perfection, but it is crucial that its application is fair and just.
- **Justice in Application** — The application of law should prioritize fairness and justice, addressing the shortcomings of the legal system.

Use via API • Built with Gradio • Settings



Adaptive, agentic study assistant.

StudyBuddy is an educational aid. Generated concepts, questions, grades, and tutor answers can contain errors — verify against your source material.

Upload / Paste Quiz Flashcards Dashboard Tutor

Ask anything about your material, or say “quiz me on X” / “what am I weak on?”

Chatbot

Explain transformers to me like I am a school kid

Transformers are like super-smart robots that help computers understand and translate languages. Imagine you have a big book in one language, and you want to read it in another language. Transformers help with that!

Here's how they work:

1. **Understanding Words Together:** Transformers look at all the words in a sentence at once, not just one by one. It's like reading a whole sentence instead of just one word at a time. This helps them understand the meaning better.
2. **Remembering Important Parts:** They have a special way to remember which words are important in a sentence. It's like highlighting the important parts in a book so you can remember them later.
3. **Two Teams - Encoder and Decoder:**
 - **Encoder Team:** This team reads the sentence in the original language and figures out what it means.
 - **Decoder Team:** This team takes the meaning and writes it out in the new language.

Message

Ask the tutor...

Send

Limitations

StudyBuddy is a learning device, not a licensed grader. It's because the mistakes of the model can get into the generated questions, grades or tutor responses, while the short answer grading may be lenient or rigorous on edge instances that outputs should be checked against the source. There are other limitations currently.

- Since OCR is not covered, scanned or image PDFs without a text layer will not be extracted.
- In the absence of a model capable of executing tool calls, the tutor may as well be chatting with plain text.
- Chroma is a local store and SQLite is a single-writer fine for a single-user demo, but not the right choice for a multi-user scale.

These limitations directly inform future work: add OCR and support for richer document types (DOCX, PPTX); switch to Postgres and a hosted vector database for multi-user, multi-class deployment; implement a sharper spaced-repetition schedule and confidence-weighted mastery; and add an evaluation harness based on LangSmith to automatically score quiz and grade quality.